Concurrency, Parallelism, Distributed Systems

Systems-First Detailed Course Syllabus

Lebanese University
Faculty of Engineering
Semester VIII
Academic Year 2025–2026

Instructor:
Dr. Mohamad Aoude

Lebanese University
Faculty of Engineering

Concurrency & Distributed Systems

Diagnosing Broken Systems: From Threads to Clusters

Primary Language:	Java
Counter-Example Week:	Python (with brief Go/Erlang contrast)
Level:	Senior Undergraduate / Graduate
Format:	Theory + Failure-Driven Labs + Evolving Project + Presentations
Semester:	VIII
Instructor:	Dr. Mohamad Aoude

Course Philosophy

This course teaches principles, not products.
Tools will change. The problems will not.
You are learning to diagnose illnesses — not memorize prescriptions.

Course Promise (Week 0)

By the end of this course, you will be able to diagnose and fix real concurrent system bugs professionally.
This is not theoretical knowledge — it is a marketable engineering skill.

Learning Outcomes

By the end of the course, students will be able to:

- Distinguish concurrency, parallelism, and distribution
- Design systems that are thread-safe, visibility-correct, deadlock-free, and live
- Use professional debugging tools (thread dumps, profilers) to diagnose failures
- Test concurrent code realistically (stress, timing control, reproduction strategy)
- Build and evolve a server from single-threaded → pooled → async → distributed
- Design distributed systems under latency, partial failure, retries, and backpressure
- Make defensible architectural decisions (threads vs async vs Kafka vs RPC vs sockets)

Semester Roadmap — What You Will Build

You will build **one system**, evolving it across the semester:

```
[ Week 3 ]   Single-threaded  ->  Thread-safe
(correctness under concurrency)
```



```
[ Week 5 ]   Thread pool + Backpressure
(bounded resources, fairness)

[ Week 8 ]   Async + Testing
(non-blocking design, reproducibility)

[ Week 14 ]  Distributed + Failure Recovery
(timeouts, retries, tracing, resilience)
```

Each stage **reuses the previous one** — no throwaway work.

> **Rule of the course:**
> *You don't move forward by adding features — you move forward by surviving failure.*

Semester Project Arc: What You'll Build (From Day 1)

A single evolving system, built in stages:

1. **Week 3 (Checkpoint 1):** Thread-safe server (correctness under concurrency)
2. **Week 5 (Checkpoint 2):** Thread-pooled server (bounded resources + fairness)
3. **Week 8 (Checkpoint 3 / Midterm):** Async server (non-blocking design + testing)
4. **Week 14 (Checkpoint 4 / Final):** Distributed cluster (timeouts, tracing, idempotency, resilience)

Core Framework: The Concurrency Scorecard (Used Weekly)

Every lab submission must include a completed scorecard:

```
[ ] Thread-safe? (no races)
[ ] Visibility guaranteed? (JMM reasoning)
[ ] Deadlock-free? (lock discipline)
[ ] Liveness guaranteed? (no starvation/livelock)
[ ] Bounded resources? (queues/pools/limits)
[ ] Failure recovery path? (timeouts/retries/fallbacks)
```

Explanation Quality Rubric (Used in All Reports)

- **Poor:** "It's synchronized / it works now."
- **Good:** "Synchronization enforces mutual exclusion on this monitor, preventing interleaving that violates invariant X."
- **Excellent:** "This establishes happens-before between operations A and B, preventing reordering/visibility issues, preserving invariant X under schedule Y; tradeoff is contention cost Z."

Policies (Because This Is an Iterative Engineering Course)

- **Late policy:** −10% per day, max 3 days late unless approved.
- **Revision policy:** Each checkpoint **can be revised once** after feedback, due **7 days after graded return**.
- **Collaboration:** Discuss ideas freely; code and reports must be original.

> **Professional rule:**
> *If someone else can lock your object, your invariants are already broken.*

— Readiness & Alignment

- Recommended review resources:
  - Java threads + basic I/O primer (instructor-provided links/chapters)
  - Basic CLI + Git workflow quickstart

Phase 1 — Shared-Memory Concurrency (Weeks 1–8)

Week 1 — Why Concurrency Exists (Pain First)

Motivation Demo

Single-threaded server: one request at a time → queueing → high latency → idle CPU during I/O.

Concepts

- Why concurrency exists
- Concurrency vs parallelism
- CPU vs cores
- I/O wait vs CPU work
- Amdahl's Law

Lab 1.1

Single-threaded vs multi-threaded **blocking** server.

Deliverables:

- Measurements (latency/throughput)
- Completed scorecard
- Short explanation using rubric

Case Study (Mini)

Redis/SQLite: when single-threaded wins (simplicity + cache locality).

Common Mistakes

- "More threads = faster"
- Confusing throughput with responsiveness

Week 2 — Threads, Lifecycle, and When NOT to Use Concurrency

Concepts

- `Thread` vs `Runnable`
- Thread states: NEW, RUNNABLE, BLOCKED, WAITING
- Scheduling myths
- When single-thread designs win (contention, predictability)

Lab 2.1

Thread creation patterns + join vs sleep.

Deliverables:

- Completed scorecard
- Short failure analysis: "why sleep isn't sync"

Pitfalls Cheat Sheet (Release 1)

- Don't use `sleep()` for synchronization
- Don't assume order/fairness

Week 3 — Synchronization & Locks (The Awakening)

Concepts

- Race conditions, critical sections
- `synchronized`, intrinsic locks
- Debug tools: `jstack`, thread dumps, VisualVM (basic)

Deadlock Taxonomy

1. Deadly Embrace
2. Resource Starvation
3. Livelock
4. Phantom Deadlock

Lab 3.1

Bank account race condition → fix with correct locking.

Lab 3.2 (Concrete lock-order scenario)

Transfer between accounts A→B and B→A

- Create deadlock intentionally
- Fix using **lock ordering by account ID**

Lab 3.3 — Deadlock + Thread Interruption (Failure Is Concurrent)

Extension to Deadlock Lab: after detecting the deadlock:

- Interrupt one of the blocked threads using `Thread.interrupt()`
- Require students to:
  - Catch `InterruptedException`
  - **Restore interrupt status** (`Thread.currentThread().interrupt()`)
  - Exit cleanly without corrupting shared state

Explicit Rule:

> Swallowing `InterruptedException` is considered a **bug**.

Deliverables:

- Thread dump before interruption
- Explanation of:
  - what interruption does *not* fix (deadlock itself)
  - why interruption must be handled explicitly
- Updated scorecard with failure handling noted

> **Planted Insight (Intentional):**
> *Failure isn't exceptional — it's concurrent.*

Deliverables (Mandatory visuals)

- **State diagram** of deadlock scenario
- Thread dump + interpretation
- Completed scorecard

Week 4 — Memory Visibility & Immutability

Concepts

- Visibility vs atomicity
- Caches, reordering, JMM
- `volatile`, happens-before
- Immutability as concurrency strategy

Lab 4.1–4.3

Broken counter / stale reads / DCL fix with `volatile`.

Lab 4.4 (Concrete immutable snapshot context)

Configuration cache updates without locks

- Shared mutable map causes visibility/race issues
- Fix using immutable snapshot + atomic swap of reference

Deliverables

- Short sequence diagram showing update/reader flow
- Completed scorecard

War Story Discussion (10 min)

Therac-25 (concurrency failure lessons).

Week 5 — `java.util.concurrent`: Safety, Liveness, Lock-Free

Concepts

- `ExecutorService`, thread pools
- `Callable`, `Future`, and timeout-aware result retrieval
- Fixed vs cached pools, rejection policies, and throughput tradeoffs
- Bounded queues + backpressure
- Atomics, CAS
- `ReentrantReadWriteLock` decision guidance:
  - Use when **read:write** $>$ **10:1** and reads are non-trivial
- `ThreadLocal` (useful vs abuse)
- Preview only: `Semaphore`, `CountDownLatch`, `CyclicBarrier`, and concurrent collections

Lab 5.1

Bounded `BlockingQueue`: full vs empty behavior (backpressure).

Lab 5.2

Thread pool sizing + bounded resource usage: fixed vs cached pools, `Callable`/`Future`, saturation, and rejection policy behavior.

Lab 5.3 (Lock-free programming required)

CAS counter vs `synchronized` counter

- Compare throughput and correctness under contention
- Use the fixed-vs-cached pool demo as the measurement bridge
- Discuss why atomics exist

Lab 5.4

Read-heavy workload: `ReentrantReadWriteLock` vs `synchronized`.

Lab 5.5 (ThreadLocal required)

Request context propagation

- Attach userID/requestID via `ThreadLocal`
- Show correct use + "leak" hazard (not clearing)

Deliverables

- Completed scorecard + liveness note (starvation risks)
- Measurement table + tradeoff paragraph

Pitfalls Cheat Sheet (Release 2)

- Don't oversize pools
- Don't use unbounded queues in production

Week 6 — Parallelism & Fork/Join (Granularity + Break-Even)

Concepts

- Task decomposition, work stealing
- ForkJoinPool subtleties
- **Pitfall:** blocking I/O inside ForkJoinPool starves workers

Lab 6.1

Fork/Join vs sequential (compute task).

Lab 6.2 (Numeric target)

Determine minimum array size where Fork/Join beats sequential

- Plot/record break-even point
- Explain coordination overhead

Lab 6.3 (Parallel streams anti-example + why)

Show slowdown and explain:

- boxing overhead
- lambda allocation
- spliterator/partitioning inefficiency
- thread management overhead

Case Study (Revisit Redis)

Redis v6+ I/O threads: where threads actually appear and why.

Deliverables

- Completed scorecard
- Break-even calculation and explanation

Week 7 — Async Design, Testing, and Anti-Patterns

Concepts

- `CompletableFuture` pipelines
- Timeouts/fallbacks
- Testing concurrent code (full treatment):
  - Stress testing: **JCStress overview and usage**
  - Deterministic replay techniques (conceptual)
  - Mock time/delays to reproduce races
  - Reproduction strategy: `Thread.yield()`, tight loops, randomized scheduling

Lab 7.1

Async service simulation + timeout/fallback.

Lab 7.2

Testing trap: passes 1000, fails at 1001. **Add reproduction strategy:** yield + iteration amplification.

Lab 7.3

JCStress-style stress test (guided).

Lab 7.4 — Fan-Out / Fan-In Async Pipeline (Required)

Objective: teach structured async composition and aggregation — not "fire-and-forget" chaos.

Scenario: a request is split into multiple independent async sub-tasks (fan-out), whose results must be combined into a single response (fan-in).

Tasks:

- Implement fan-out using `CompletableFuture.supplyAsync`
- Combine results using:
  - `allOf()` for synchronization
  - Explicit result aggregation
- Inject:
  - One slow task
  - One failing task
- Add timeout and fallback for the whole pipeline

Deliverables:

- Sequence diagram of fan-out / fan-in flow
- Explanation of:
  - how completion is coordinated
  - how failure propagates
- Completed scorecard

> **Core realization:**
> *Async systems are pipelines, not threads with callbacks.*

Lab 7.5 (Required: NIO counterpoint)

Single-threaded async I/O (Java NIO) vs multi-threaded blocking

- Same hardware
- Measure throughput/latency
- Prove: async $\neq$ more threads

Anti-Patterns Module (with mini-examples)

- Thread-per-request
- Unbounded queues
- Missing timeouts
- Silent failure swallowing

Deliverables

- Completed scorecard
- Sequence diagram of async pipeline

Week 8 — Midterm: Debugging + Kafka Mental Model Preview

Midterm Submission (Tool usage required)

- Thread dump
- Diagnosis report
- Fix
- Performance tradeoff analysis
- 5-minute recorded explanation to technical audience

Rubric: 1) real bug, 2) mechanism, 3) cost.

Kafka Preview (20-minute lecture, no lab)

"Logs are the universal abstraction."

- Log as history, replay, recovery, audit, decoupling
- Why distributed logs show up everywhere (Kafka, event sourcing)

Pitfalls Cheat Sheet (Release 3)

- Measure before optimizing
- Async $\neq$ faster
- Fix mechanism, not symptom

Phase 2 — Language Models & Limits (Week 9)

Week 9 — Python GIL + Brief Survey of Alternatives

Concepts

- GIL reality: CPU-bound threads don't scale
- Multiprocessing tradeoffs
- 15-minute contrast:
  - Go goroutines (scheduler + channels)
  - Erlang actors (message passing model)

Lab 9.1

Python threads vs multiprocessing (CPU-bound).

Lab 9.2 (Visual proof)

`htop` / `py-spy`:

- 4 threads → 1 core active in Python
- 4 Java threads → multiple cores

War Story Discussion (10 min)

Knight Capital (timing + deployment + race-type failures).

Phase 3 — Distributed Systems (Weeks 10–14)

Week 10 — Network = Concurrency + Failure (Phase 3 Start)

Concepts

- Networks are slow queues with partial failure
- No shared memory
- Retry amplifies load

Three Laws of Networks

1. Latency is never zero
2. Bandwidth is never infinite
3. Failures are partial and invisible

Tool Introduction (Required): Toxiproxy

Used in Weeks 10–13:

- latency injection
- bandwidth limit
- connection reset/timeouts

Lab 10.1

Local `BlockingQueue` → socket queue (same business logic). Then use Toxiproxy to inject:

- latency
- packet loss/reset

Deliverables:

- Completed scorecard
- Measurement summary

Week 11 — Client–Server, Serialization, Versioning + Peer Review

Concepts

- TCP sockets
- serialization boundary = no shared objects
- defensive copying, immutability
- versioning failures

Lab 11.1

Multi-client server, immutability fix.

Lab 11.2 (Versioning: specify how)

Choose one:

- Java serialization with `serialVersionUID` and compatibility handling, OR
- Protocol Buffers with field evolution (recommended)

Peer Review Checkpoint (Required)

Students swap Week 10 socket code:

- Find concurrency bug or failure scenario
- Provide written review (what breaks, evidence, suggested fix)

Deliverables

- Completed scorecard
- **Sequence diagram** showing message flow

Pitfalls Cheat Sheet (Release 4)

- Don't assume network reliability
- Always time out network calls

Week 12 — Kafka: Partitions, Consumer Groups, Backpressure

Concepts

- Kafka as distributed log (not "Kafka course")
- Partitions: ordering vs parallelism
- Consumer groups + rebalancing
- Backpressure and lag

Lab 12.1

Partitions + same key routing + ordering effects.

Lab 12.2 (Consumer groups)

2 consumers, 4 partitions:

- observe assignment
- trigger rebalance and observe behavior

Lab 12.3 (Backpressure)

Flood topic with 10k msg/s, 1 slow consumer:

- measure lag growth
- discuss mitigation (more partitions, faster consumers, batch, retention)

Partition Decision Exercise (Constrained)

Given:

- 1M msg/s
- 3 consumers
- 100ms max lag target

Students must propose partition count with calculations and assumptions.

Case Study

Kafka at LinkedIn (design motivations).

Week 13 — RPC, Cascading Failure, Tracing, Idempotency, Clock Drift

Concepts

- gRPC: contracts and timeouts
- retries and their dangers
- **idempotency:** retries require idempotent operations
- distributed tracing without fancy tools: request IDs + log correlation
- log aggregation basics: grep across logs + timestamp correlation
- clock drift: time is the enemy

Circuit Breaker (No instructor dependency)

Students implement using one of:

- **Resilience4j** (recommended), OR
- Pseudocode spec (state machine: closed/open/half-open)

Lab 13.1 — Cascading Failure (Explicit)

1. Healthy baseline (measure)
2. Inject 10% timeouts via Toxiproxy
3. Observe thread pool exhaustion
4. Trace failure propagation
5. Add circuit breaker
6. Add idempotency fix for retry-safe operations

Lab 13.2 — Tracing Reconstruction

Log trace through 3-service chain:

- propagate request IDs
- reconstruct timeline from logs (manual trace analysis)

Lab 13.3 — Clock Drift Demo

Skew clocks; show timestamp ordering violation and why correlation must be careful.

Week 14 — Final Synthesis, Presentations, Tool Evaluation

Portfolio (80%) — Checkpoint Weights

- Week 3: **10%**
- Week 5: **15%**
- Week 8 (Midterm): **25%**
- Week 14 (Final): **30%**

Final must include:

- explicit failure guarantees (timeouts, retry behavior)
- tracing evidence
- idempotency handling
- completed scorecard

Architecture Decision Memo (20%) — Rubric

- Correctness of tradeoff analysis: **40%**
- Justification of choice: **30%**
- Consideration of failure modes: **20%**
- Clarity: **10%**

Memo uses decision matrix:

- latency requirements
- ordering guarantees
- failure modes
- operational complexity

Presentation (Week 14)

5 minutes to a technical audience:

- explain one failure from your distributed system
- show evidence (logs/traces/metrics)
- explain fix and tradeoff

Further Study (Specified)

Annotated bibliography (3–5 items each):

- Actor models
- CRDTs
- Consensus (Raft/Paxos)
- Advanced tracing/observability
- Formal concurrency testing

Tool Evaluation Framework (Operationalizing "tools change")

Checklist for evaluating any new concurrency tool/framework:

- What guarantees does it provide?
- What failure modes does it hide or introduce?
- What is the mental model?
- How does it behave under overload?
- How do you debug it?

War Story Discussion (10 min)

AWS S3 outage synthesis + "what would you instrument?"

Case Studies + Discussion Component (Required)

- Phase 1: Therac-25 (Week 4 discussion)
- Phase 2: Knight Capital (Week 9 discussion)
- Phase 3: AWS S3 outage (Week 14 discussion)

Progressive Visual Requirements

- Week 3: **state diagram** (deadlock)
- Week 8: **sequence diagram** (async pipeline)
- Week 14: **system sequence + failure propagation diagram**

Final Statement

This course does not train you to "use concurrency libraries." It trains you to **diagnose broken systems**.

When your future production system freezes, corrupts data, or collapses under load, you will not panic. You will take a thread dump, trace the failure, identify the mechanism, and fix it professionally.

